BÀI THỰC HÀNH XỬ LÝ TÍN HIỆU SỐ

Trần Duy Tiến - 2413488
Võ Chí Cương - 2410420
9/11/2026

Tổng Quan

Báo cáo này trình bày tổng quát quá trình thực hành và ứng dụng phần mềm Scilab trong môn học Xử lý tín hiệu số. Nội dung cốt lõi xoay quanh việc lập trình và mô phỏng các phép toán cơ bản trên tín hiệu rời rạc, bao gồm:

- **Dịch trễ (Delay) và Dịch sớm (Advance):** Thay đổi vị trí tương đối của tín hiệu, dịch chuyển dọc theo trục thời gian.
- **Đảo thời gian (Fold):** Phản chiếu tín hiệu qua gốc tọa độ trục tung để tạo phiên bản đối xứng của chính nó.
- **Cộng tín hiệu (Add):** Tổng hợp biên độ của hai tín hiệu rời rạc tại từng mốc thời gian tương ứng.
- **Tích chập (Convolution):** Xác định đáp ứng đầu ra của hệ thống dựa trên sự kết hợp giữa tín hiệu đầu vào và đáp ứng xung.

Bên cạnh việc giải thích cơ sở lý thuyết, xây dựng mã nguồn Scilab và trực quan hóa kết quả bằng đồ thị, báo cáo cũng cung cấp lời giải toán học chi tiết cho các bài tập lý thuyết bổ trợ nhằm củng cố kiến thức một cách toàn diện.

1 Giới Thiệu

Xử lý tín hiệu số là một lĩnh vực nền tảng đòi hỏi sự kết hợp chặt chẽ giữa lý thuyết toán học và công cụ mô phỏng thực tế. Trong bài báo cáo này, phần mềm Scilab được sử dụng như một môi trường lập trình chủ đạo để trực tiếp xử lý các phép toán cơ bản trên tín hiệu rời rạc. Thông qua việc tự tay xây dựng các hàm chức năng (delay, advance, fold, add, convolution) thay vì dùng hàm có sẵn, bài thực hành không chỉ giúp làm rõ bản chất của các hệ thống rời rạc mà còn rèn luyện tư duy thuật toán và nâng cao kỹ năng lập trình khoa học trên nền tảng Scilab.

2 Mục Tiêu

- Nắm vững cú pháp, nguyên lý hoạt động và cách ứng dụng các hàm cơ bản trong môi trường lập trình Scilab.
- Hiểu sâu sắc bản chất toán học của các phép biến đổi và phép toán trên tín hiệu thời gian rời rạc.
- Rèn luyện kỹ năng chuyển đổi từ công thức lý thuyết sang mã lệnh thực thi một cách chính xác.
- Biểu diễn trực quan dữ liệu tín hiệu bằng cách sử dụng các công cụ vẽ đồ thị của Scilab.
- Đánh giá và kiểm chứng tính đúng đắn của các thuật toán thông qua việc thử nghiệm với các ví dụ cụ thể.
- Hoàn thiện và giải quyết triệt để các bài tập lý thuyết bổ sung để nắm bắt trọn vẹn nội dung chương học.

3 Phương Pháp

Để đạt được các mục tiêu đề ra, quá trình thực hành được triển khai theo phương pháp đối chiếu song song giữa mô phỏng và lý thuyết:

- **Xây dựng thuật toán:** Viết các tập lệnh và định nghĩa hàm chức năng trong Scilab để thực thi từng phép toán xử lý tín hiệu số.
- **Trực quan hóa:** Sử dụng bộ lệnh đồ họa của Scilab để vẽ trục thời gian và biên độ, giúp quan sát trực tiếp sự biến đổi của tín hiệu trước và sau khi xử lý.
- **Kiểm chứng:** Chạy thử nghiệm các đoạn mã với các tín hiệu mẫu đầu vào, sau đó đối chiếu kết quả mô phỏng trên đồ thị với kết quả tính toán giải tích bằng tay để xác minh tính chính xác tuyệt đối.

4 Nội Dung Thực Hành

Exercise 1

Tín hiệu đầu vào:

$$x(n) = 2\delta(n+2) - 1\delta(n+1) + 2\delta(n) - 3\delta(n-1) + 4\delta(n-2)$$

Biến đổi Z (Z-Transform):

Áp dụng tính chất dịch thời gian của biến đổi Z đối với hàm xung đơn vị $\mathcal{Z}\{\delta(n-k)\} = z^{-k}$, ta phân tích từng thành phần của tín hiệu:

- $\mathcal{Z}\{2\delta(n+2)\} = 2z^2$
- $\mathcal{Z}\{-1\delta(n+1)\} = -z^1$
- $\mathcal{Z}\{2\delta(n)\} = 2z^0 = 2$
- $\mathcal{Z}\{-3\delta(n-1)\} = -3z^{-1}$
- $\mathcal{Z}\{4\delta(n-2)\} = 4z^{-2}$

Cộng các thành phần tuyến tính này lại, biểu thức biến đổi Z của tín hiệu là:

$$X(z) = 2z^2 - z + 2 - 3z^{-1} + 4z^{-2}$$

Miền hội tụ (Region of Convergence - ROC):

Để xác định miền hội tụ, ta xét điều kiện tồn tại (không tiến tới vô cùng) của các số hạng trong biểu thức $X(z)$:

- Các thành phần chứa lũy thừa dương của z $(2z^2, -z)$ sẽ phân kỳ khi $z \to \infty$. Do đó, chúng hội tụ với điều kiện $z \neq \infty$.
- Các thành phần chứa lũy thừa âm của z $(-3z^{-1}, 4z^{-2})$ tương đương với việc chia cho z, sẽ phân kỳ khi $z = 0$. Do đó, chúng hội tụ với điều kiện $z \neq 0$.

Lấy phần giao của các điều kiện trên, ta có miền hội tụ chung của toàn bộ tín hiệu là toàn bộ mặt phẳng phức z, ngoại trừ gốc tọa độ và điểm vô cực.

ROC: $0 < |z| < \infty$

Exercise 2

Tín hiệu đầu vào:

$$x(n) = 0.5^n u(n) + 0.4^n u(n)$$

Biến đổi Z (Z-Transform):

Áp dụng công thức biến đổi Z tiêu chuẩn cho chuỗi nhân quả $\mathcal{Z}\{a^n u(n)\} = \frac{1}{1-az^{-1}}$, ta phân tích từng thành phần của tín hiệu:

- $\mathcal{Z}\{0.5^n u(n)\} = \frac{1}{1-0.5z^{-1}}$
- $\mathcal{Z}\{0.4^n u(n)\} = \frac{1}{1-0.4z^{-1}}$

Cộng hai thành phần tuyến tính này lại, ta có biểu thức biến đổi Z:

$$X(z) = \frac{1}{1-0.5z^{-1}} + \frac{1}{1-0.4z^{-1}}$$

Tiến hành quy đồng mẫu số để rút gọn biểu thức:

$$X(z) = \frac{(1-0.4z^{-1}) + (1-0.5z^{-1})}{(1-0.5z^{-1})(1-0.4z^{-1})}$$

$$X(z) = \frac{2-0.9z^{-1}}{1-0.9z^{-1}+0.2z^{-2}}$$

Miền hội tụ (Region of Convergence - ROC):
Miền hội tụ của tín hiệu tổng hợp là phần giao (intersection) của các miền hội tụ thành phần:

- Thành phần $0.5^n u(n)$ hội tụ với điều kiện $|z| > 0.5$.
- Thành phần $0.4^n u(n)$ hội tụ với điều kiện $|z| > 0.4$.

Lấy phần giao của hai điều kiện bất phương trình trên, miền hội tụ chung bị giới hạn bởi vòng tròn có bán kính lớn hơn.
ROC: $|z| > 0.5$

Exercise 3

Tín hiệu đầu vào:

$$x(n) = 0.5^n u(n) + 0.9^n u(-n-1)$$

Bước 1: Tính biến đổi Z cho từng thành phần (Z-Transform)
Tín hiệu được chia làm hai chuỗi cơ bản để tính toán trực tiếp:

- **Thành phần nhân quả (causal sequence):** $x_1(n) = 0.5^n u(n)$
  - Áp dụng cặp biến đổi Z chuẩn $\mathcal{Z}\{a^n u(n)\} = \frac{1}{1-az^{-1}}$, ta có:

$$X_1(z) = \frac{1}{1-0.5z^{-1}}$$

  - Miền hội tụ của phần này nằm ngoài đường tròn bán kính 0.5: $|z| > 0.5$.

- **Thành phần phản nhân quả (anti-causal sequence):** $x_2(n) = 0.9^n u(-n-1)$
  - Từ cặp biến đổi Z chuẩn $\mathcal{Z}\{-a^n u(-n-1)\} = \frac{1}{1-az^{-1}}$, suy ra $\mathcal{Z}\{a^n u(-n-1)\} = -\frac{1}{1-az^{-1}}$, ta có:

$$X_2(z) = -\frac{1}{1-0.9z^{-1}}$$

  - Miền hội tụ của phần này nằm trong đường tròn bán kính 0.9: $|z| < 0.9$.

Bước 2: Tổng hợp biến đổi Z toàn tín hiệu
Cộng tuyến tính hai thành phần trên lại:

$$X(z) = X_1(z) + X_2(z) = \frac{1}{1-0.5z^{-1}} - \frac{1}{1-0.9z^{-1}}$$

Tiến hành quy đồng mẫu số để rút gọn về dạng phân thức đơn giản:

$$X(z) = \frac{(1-0.9z^{-1}) - (1-0.5z^{-1})}{(1-0.5z^{-1})(1-0.9z^{-1})}$$

$$X(z) = \frac{-0.4z^{-1}}{1-1.4z^{-1}+0.45z^{-2}}$$

Bước 3: Xác định miền hội tụ chung (ROC)
Miền hội tụ (ROC) của tín hiệu tổng hợp là phần giao (intersection) của các ROC thành phần. Ta cần thỏa mãn đồng thời hai điều kiện $|z| > 0.5$ và $|z| < 0.9$.
ROC: $0.5 < |z| < 0.9$ (Tạo thành một hình vành khăn hội tụ trên mặt phẳng phức Z).

Exercise 4

Nghiên cứu (Investigate):
Trong Scilab, tín hiệu âm thanh rời rạc $x(n)$ được quản lý và biểu diễn dưới dạng ma trận 1 chiều chứa các giá trị biên độ theo thời gian. Scilab tích hợp sẵn thư viện Sound File Handling trong module lõi với các hàm chính:

- `wavread()`: Đọc file âm thanh `.wav` thành ma trận tín hiệu và trả về tần số lấy mẫu F_s (Sampling rate).
- `wavwrite()`: Ghi ma trận tín hiệu âm thanh ra file định dạng `.wav`.
- `playsnd()`: Phát ma trận tín hiệu âm thanh ra thiết bị phát (loa).
- `analyze()`: Phân tích phổ tần số của tín hiệu âm thanh trên miền tần số.

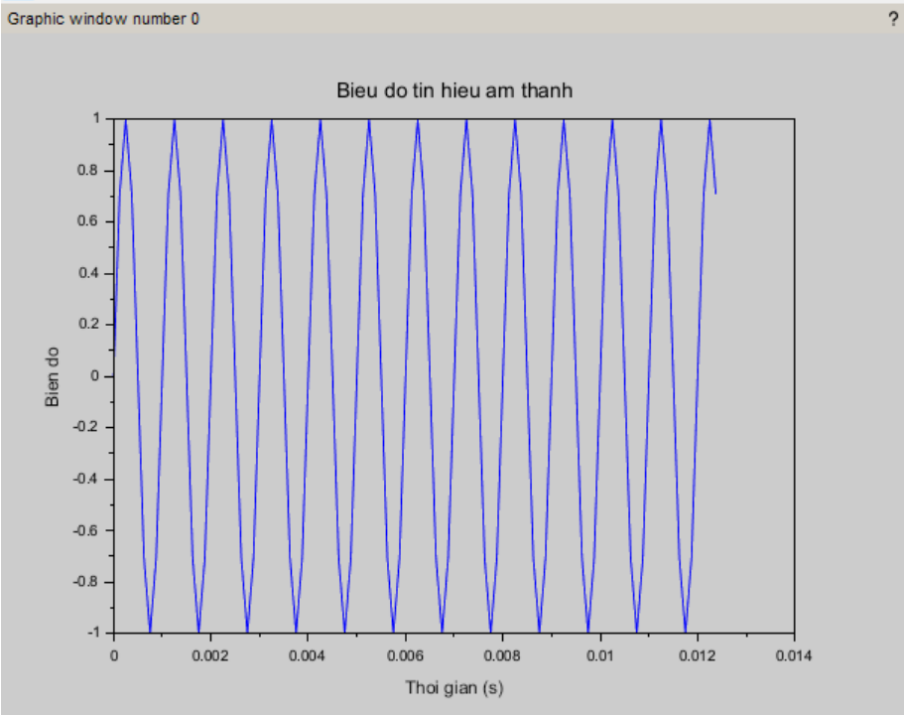
Chương trình minh họa (Scilab Script):

```
// ========================================
// LAB 04 - EXERCISE 4: AUDIO SIGNAL PROCESSING
// ========================================
clear; clc; close(winsid());

// 1. Tao tin hieu am thanh song sin roi rac x(n)
Fs = 8000;                  // Tan so lay mau (Hz)
t = 0:1/Fs:2;               // Truc thoi gian (2 giay)
f0 = 1000;                  // Tan so am thanh (1000 Hz)
x = sin(2 * %pi * f0 * t);  // Tin hieu am thanh roi rac

// 2. Phat am thanh ra loa
playsnd(x, Fs);

// 3. Hien thi dang song tin hieu am thanh tren mien thoi gian
figure(1);
plot(t(1:100), x(1:100));
xtitle("Dang song tin hieu am thanh x(n) (100 mau dau)", "Thoi gian
    (s)", "Bien do");
xgrid();
```



Exercise 5

Nghiên cứu thư viện xử lý ảnh (IPCV Toolbox):

Thư viện IPCV (Image Processing and Computer Vision) trong Scilab cung cấp đầy đủ các công cụ xử lý ảnh và thị giác máy tính:

- Đọc, hiển thị và chuyển đổi không gian màu (RGB sang Grayscale).
- Tính toán, hiển thị biểu đồ mức xám (Histogram) và cân bằng độ tương phản (Histogram Equalization).
- Lọc không gian, xoay, thay đổi kích thước và làm mờ ảnh (Image Blurring).

Cài đặt và nạp thư viện trong Scilab Console:

```
atomsInstall("IPCV");
atomsLoad("IPCV");
```

Chương trình minh họa (Scilab Script):

```
clc; clear; close;

// 1. Doc anh va chuyen sang anh xam
img = imread("input.jpg");
gray = rgb2gray(img);

// 2. Tinh Histogram ban dau
H1 = imhist(gray);

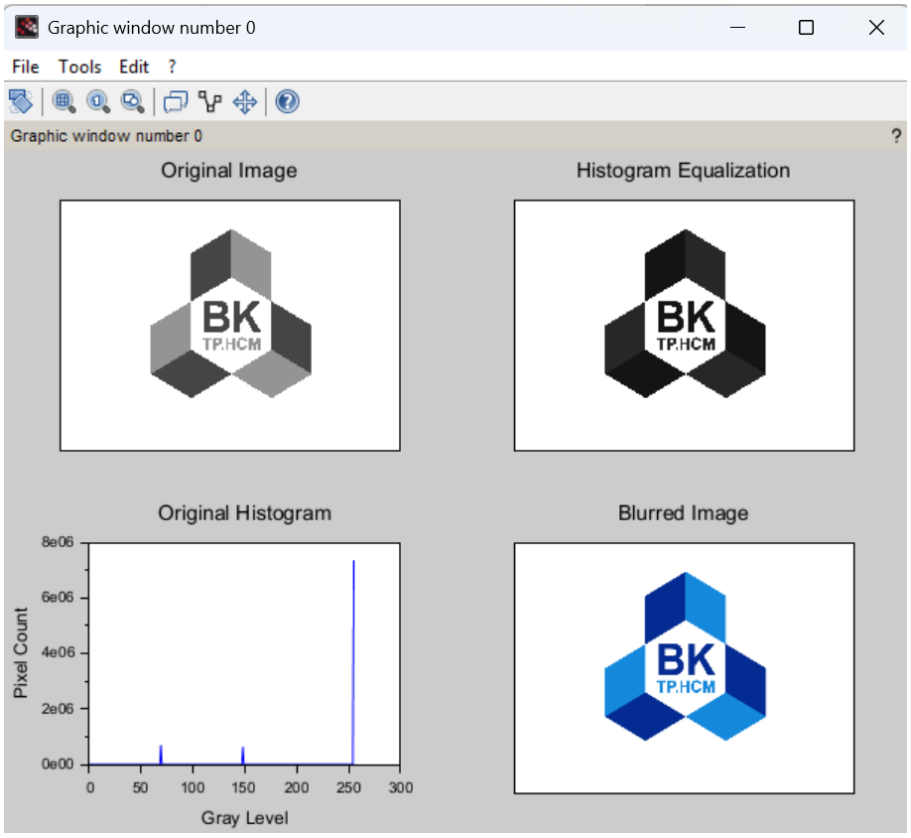
// 3. Can bang Histogram (Histogram Equalization)
CDF = cumsum(H1);
CDF = CDF / CDF($);
LUT = uint8(round(255 * CDF));
equalized = LUT(double(gray) + 1);

// 4. Lam mo anh bang bo loc trung binh 5x5
kernel = fspecial("average", [5 5]);
blurred = imfilter(img, kernel);

// 5. Hien thi 4 ket qua tren khung Subplot 2x2
figure();
subplot(2,2,1); imshow(gray); title("Original Image");
subplot(2,2,2); imshow(equalized); title("Histogram Equalization");
subplot(2,2,3); plot(0:255, H1'); title("Original Histogram");
   xlabel("Gray Level"); ylabel("Pixel Count");
subplot(2,2,4); imshow(blurred); title("Blurred Image");
```

Nhận xét kết quả:

- **Original Image:** Ảnh xám gốc đầu vào.
- **Histogram Equalization:** Ảnh sau khi cân bằng histogram, giúp làm nổi bật đường nét và cải thiện độ tương phản rõ rệt.
- **Original Histogram:** Biểu đồ thể hiện mật độ phân bố các mức xám (từ 0 đến 255) của ảnh ban đầu.
- **Blurred Image:** Ảnh sau khi áp dụng bộ lọc trung bình 5×5, giúp làm mượt nét và giảm bớt các chi tiết nhiễu.



Exercise 6

Đề bài: Cho hai tín hiệu rời rạc $x(n)$ và $h(n)$. Từ định nghĩa

$$y(n) = \sum_{k=-\infty}^{+\infty} h(k)x(n-k),$$

hãy rút gọn biểu thức cho hai trường hợp: cả hai tín hiệu nhân quả và đáp ứng xung hữu hạn (FIR). Với $x(n) = [\underset{n=0}{\uparrow} 1, 2, -3, 2, 1]$ và $h(n) = [\underset{n=0}{\uparrow} 1, 0, -1]$, hãy tính tích chập bằng phương pháp gấp–dịch và phương pháp ma trận; viết mã Scilab, vẽ tín hiệu và năng lượng.

1. Cơ sở lý thuyết

Trường hợp hai tín hiệu nhân quả: $h(k) = 0$ nếu $k < 0$ và $x(n-k) = 0$ khi $n - k < 0$. Vì vậy, đối với $n \geq 0$, chỉ những chỉ số $0 \leq k \leq n$ có đóng góp vào tổng:

$$y(n) = \begin{cases} \displaystyle\sum_{k=0}^{n} h(k)x(n-k), & n \geq 0, \\ 0, & n < 0. \end{cases}$$

Với mọi n cố định, đây là tổng hữu hạn nên xác định nếu các mẫu tín hiệu đều hữu hạn.

Trường hợp đáp ứng xung hữu hạn: Nếu $h(k) = 0$ với $k < 0$ hoặc $k > M$, thì

$$\boxed{y(n) = \sum_{k=0}^{M} h(k)x(n-k)}.$$

Do tổng chỉ gồm $M + 1$ phần tử, $y(n)$ xác định tại mỗi n khi các giá trị $x(n-k)$ hữu hạn. Bài toán cho $M = 2$.

2. Phương pháp gấp và dịch (Folding and Shifting)

Quá trình chập gồm gấp $x(k)$ thành $x(-k)$, dịch thành $x(n-k)$, nhân từng mẫu với $h(k)$ rồi cộng lại. Vì $h(0) = 1$, $h(1) = 0$, $h(2) = -1$, ta được:

$$y(n) = x(n) - x(n-2).$$

Quy ước $x(n) = 0$ khi $n < 0$ hoặc $n > 4$. Tính lần lượt:

n	Phép tính $x(n) - x(n-2)$	$y(n)$
0	$1 - 0$	1
1	$2 - 0$	2
2	$-3 - 1$	-4
3	$2 - 2$	0
4	$1 - (-3)$	4
5	$0 - 2$	-2
6	$0 - 1$	-1

Suy ra tín hiệu đầu ra có $5 + 3 - 1 = 7$ mẫu:

$$\boxed{y(n) = [\underset{n=0}{\uparrow} 1, 2, -4, 0, 4, -2, -1]}.$$

3. Phương pháp ma trận (Matrix Method)

Tích chập tuyến tính có thể biểu diễn bằng phép nhân giữa ma trận Toeplitz của $h(n)$ với vectơ x:

$$\mathbf{H}_{\text{lin}} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 \\ 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & -1 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -3 \\ 2 \\ 1 \end{bmatrix}.$$

$$\mathbf{y} = \mathbf{H}_{\text{lin}}\mathbf{x} = \begin{bmatrix} 1 \\ 2 \\ -4 \\ 0 \\ 4 \\ -2 \\ -1 \end{bmatrix}.$$

Kết quả này trùng khớp với phương pháp gấp–dịch và được kiểm tra lại bằng hàm `conv()` của Scilab.

4. Năng lượng tín hiệu

Năng lượng của một tín hiệu có hữu hạn mẫu được tính bởi $E_x = \sum_n |x(n)|^2$. Do đó:

$$E_x = 1^2 + 2^2 + (-3)^2 + 2^2 + 1^2 = \boxed{19},$$

$$E_y = 1^2 + 2^2 + (-4)^2 + 0^2 + 4^2 + (-2)^2 + (-1)^2 = \boxed{42}.$$

Đồ thị hiển thị đồng thời $x(n)$, $y(n)$, năng lượng từng mẫu $|x(n)|^2$ và $|y(n)|^2$ trong một cửa sổ gồm bốn ô.

5. Chương trình minh họa (Scilab Script):

```
// ================================================
// LAB 4 - EXERCISE 6: LINEAR CONVOLUTION
// ================================================
clear; clc; close(winsid());

 // 1. Input signals
 x = [1, 2, -3, 2, 1];
 h = [1, 0, -1];

 N = length(x);
 M = length(h);
 L = N + M - 1;

 nx = 0:N-1;
 ny = 0:L-1;

 // 2. Folding and shifting method
 y_fold = zeros(1, L);

 for n = 0:L-1
     for k = 0:M-1
         j = n-k;

         if j >= 0 & j < N then
             y_fold(n+1) = y_fold(n+1) + h(k+1)*x(j+1);
         end
     end
 end

 disp("Exercise 6 - Folding/shifting result:");
 disp(y_fold);

 // 3. Matrix method
 H = zeros(L, N);

 for i = 1:M
     for j = 1:N
         H(i+j-1, j) = h(i);
     end
 end

 y_matrix = (H*x')';

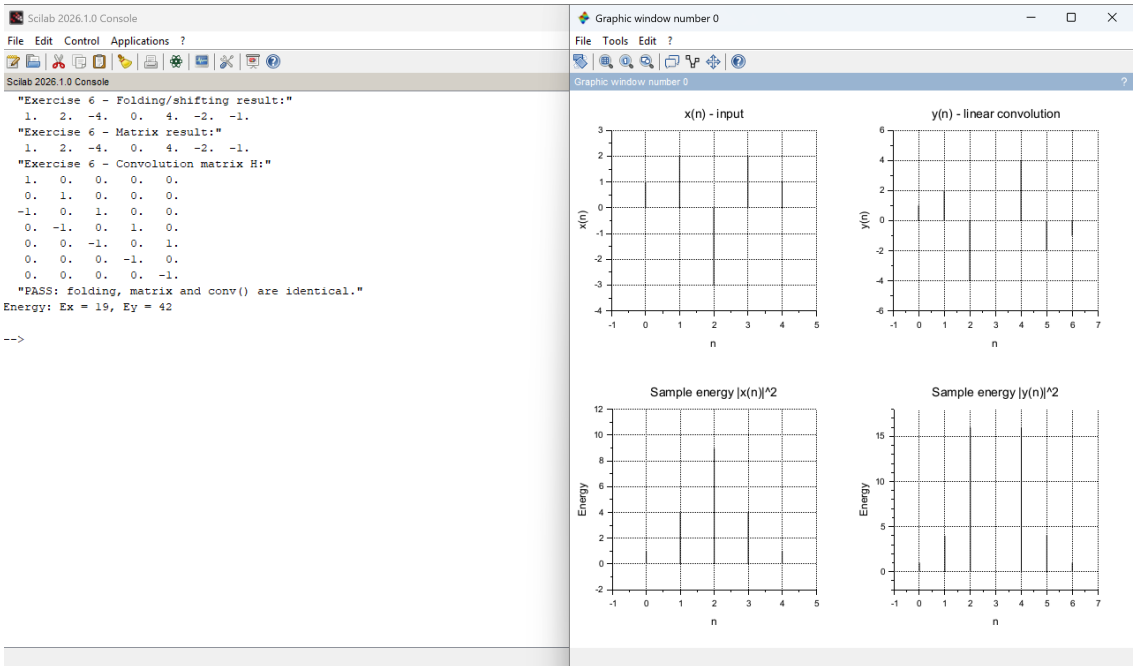
 disp("Exercise 6 - Matrix result:");
 disp(y_matrix);
```

```
46
47 disp("Exercise 6 - Convolution matrix H:");
48 disp(H);
49
50 // 4. Verify using built-in convolution
51 y_check = conv(x, h);
52
53 if max(abs(y_fold-y_matrix)) < 1d-10 & ...
54    max(abs(y_fold-y_check)) < 1d-10 then
55     disp("PASS: folding, matrix and conv() are identical.");
56 else
57     disp("FAIL: convolution results do not match.");
58 end
59
60 // 5. Signal energy
61 ex = abs(x).^2;
62 ey = abs(y_fold).^2;
63
64 Ex = sum(ex);
65 Ey = sum(ey);
66
67 mprintf("Energy: Ex = %.0f, Ey = %.0f\n", Ex, Ey);
68
69 // 6. Plot all signals in one figure
70 scf(0);
71 clf();
72
73 // Input x(n)
74 subplot(2,2,1);
75 plot2d3(nx', x');
76 xtitle("x(n) - input", "n", "x(n)");
77 a = gca();
78 a.data_bounds = [-0.5, -4; 4.5, 3];
79 xgrid();
80
81 // Linear convolution y(n)
82 subplot(2,2,2);
83 plot2d3(ny', y_fold');
84 xtitle("y(n) - linear convolution", "n", "y(n)");
85 a = gca();
86 a.data_bounds = [-0.5, -5; 6.5, 5];
87 xgrid();
88
89 // Energy |x(n)|^2
90 subplot(2,2,3);
91 plot2d3(nx', ex');
92 xtitle("Sample energy |x(n)|^2", "n", "Energy");
93 a = gca();
94 a.data_bounds = [-0.5, -1; 4.5, 11];
95 xgrid();
96
97 // Energy |y(n)|^2
98 subplot(2,2,4);
99 plot2d3(ny', ey');
100 xtitle("Sample energy |y(n)|^2", "n", "Energy");
101 a = gca();
102 a.data_bounds = [-0.5, -1; 6.5, 18];
```

```
103 xgrid();
```

Listing 1: Mã Scilab cho Exercise 6 – tích chập tuyến tính

6. Kết quả kiểm thử trên Scilab



Hình 1: Exercise 6: kết quả Console

Nhận xét: Hai phương pháp tính trực tiếp và ma trận đều cho $y(n) = [1, 2, -4, 0, 4, -2, -1]$; kết quả cũng được đối chiếu với hàm `conv()` trong Scilab.

Exercise 7

Đề bài: Xét $x(n)$ tuần hoàn có chu kỳ N và đáp ứng xung $h(n)$ gồm M mẫu. Giải thích công thức tích chập vòng, sau đó tính $y(n)$ bằng phương pháp gấp–dịch và ma trận với

$$x(n) = [\underset{n=0}{\uparrow} 1, 2, -3, 2, 1], \qquad h(n) = [\underset{n=0}{\uparrow} 1, 0, -1, -1, 1].$$

Viết mã Scilab và biểu diễn đồ thị tín hiệu cùng năng lượng.

1. Giải thích công thức tích chập vòng

Với tín hiệu có chu kỳ N, mọi chỉ số được hiểu theo modulo N:

$$x(n+N) = x(n), \qquad x(n-k) = x\big[(n-k) \bmod N\big].$$

Thay biểu thức tuần hoàn này vào công thức chập tuyến tính và sử dụng tính chất $h(k) = 0$ ngoài đoạn $0 \le k \le M-1$, ta được:

$$\boxed{y(n) = \sum_{k=0}^{M-1} h(k)\, x\big[(n-k) \bmod N\big].}$$

Phép modulo làm cho chỉ số vượt ra ngoài một chu kỳ quay trở lại đoạn $0, 1, \ldots, N-1$, nên $y(n)$ cũng có chu kỳ N. Trong bài này $N = M = 5$.

2. Phương pháp gấp và dịch vòng (Folding and Shifting)

Từ $h(n) = [1, 0, -1, -1, 1]$, ta có:

$$y(n) = x[n]_5 - x[n-2]_5 - x[n-3]_5 + x[n-4]_5,$$

trong đó $x[r]_5 = x(r \bmod 5)$. Thực hiện gấp và dịch tuần hoàn từng mẫu:

n	Các thành phần trong tổng	$y(n)$
0	$1 - 2 - (-3) + 2$	4
1	$2 - 1 - 2 + (-3)$	-4
2	$-3 - 1 - 1 + 2$	-3
3	$2 - 2 - 1 + 1$	0
4	$1 - (-3) - 2 + 1$	3

Vậy một chu kỳ của kết quả tích chập vòng là:

$$\boxed{y(n) = [\underset{n=0}{\uparrow} 4, -4, -3, 0, 3]}.$$

Chú ý: đối chiếu trực tiếp công thức xác nhận $y(3) = 2 - 2 - 1 + 1 = 0$.

3. Phương pháp ma trận (Matrix Method)

Tạo ma trận tuần hoàn $\mathbf{H}_{\text{circ}}$ kích thước 5×5 từ $h(n)$; mỗi hàng là một phép dịch vòng của hàng trước:

$$\mathbf{H}_{\text{circ}} = \begin{bmatrix} 1 & 1 & -1 & -1 & 0 \\ 0 & 1 & 1 & -1 & -1 \\ -1 & 0 & 1 & 1 & -1 \\ -1 & -1 & 0 & 1 & 1 \\ 1 & -1 & -1 & 0 & 1 \end{bmatrix}.$$

$$\mathbf{y} = \mathbf{H}_{\text{circ}} \begin{bmatrix} 1 \\ 2 \\ -3 \\ 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ -4 \\ -3 \\ 0 \\ 3 \end{bmatrix}.$$

Phương pháp ma trận cho cùng kết quả với phương pháp gấp–dịch vòng.

4. Năng lượng tín hiệu trên một chu kỳ

Do $x(n)$ và $y(n)$ được xét tuần hoàn, năng lượng dưới đây được tính *trong một chu kỳ* gồm 5 mẫu (nếu lặp trên toàn bộ trục thời gian, năng lượng toàn phần của tín hiệu tuần hoàn khác 0 sẽ vô hạn):

$$E_x^{(N)} = \sum_{n=0}^{4} |x(n)|^2 = \boxed{19},$$

$$E_y^{(N)} = \sum_{n=0}^{4} |y(n)|^2 = 4^2 + (-4)^2 + (-3)^2 + 0^2 + 3^2 = \boxed{50}.$$

5. Chương trình minh họa (Scilab Script):

```
// ================================================
// LAB 4 - EXERCISE 7: CIRCULAR CONVOLUTION
// ================================================
clear; clc; close(winsid());

 // 1. Input signals
 x = [1, 2, -3, 2, 1];
 h = [1, 0, -1, -1, 1];

 N = length(x);
 M = length(h);

 n_axis = 0:N-1;

 // 2. Folding and shifting method
 y_fold = zeros(1, N);

 for n = 0:N-1
     for k = 0:M-1
         j = pmodulo(n-k, N);
         y_fold(n+1) = y_fold(n+1) + h(k+1)*x(j+1);
     end
 end

 disp("Exercise 7 - Folding/shifting result:");
 disp(y_fold);

 // 3. Matrix method
 H = zeros(N, N);

 for n = 0:N-1
     for k = 0:M-1
         j = pmodulo(n-k, N);
         H(n+1, j+1) = H(n+1, j+1) + h(k+1);
     end
 end

 y_matrix = (H*x')';

 disp("Exercise 7 - Matrix result:");
 disp(y_matrix);

 disp("Exercise 7 - Circular convolution matrix H:");
 disp(H);

 // 4. Verify using linear convolution
 y_linear = conv(x, h);
 y_check = zeros(1, N);

 for i = 1:length(y_linear)
     j = pmodulo(i-1, N) + 1;
     y_check(j) = y_check(j) + y_linear(i);
 end

 if max(abs(y_fold-y_matrix)) < 1d-10 & ...
    max(abs(y_fold-y_check)) < 1d-10 then
     disp("PASS: folding, matrix and circular check are identical.");
 else
     disp("FAIL: circular convolution results do not match.");
 end

 // 5. Signal energy (one period)
 ex = abs(x).^2;
 ey = abs(y_fold).^2;

 Ex = sum(ex);
 Ey = sum(ey);

 mprintf("Energy: Ex = %.0f, Ey = %.0f\n", Ex, Ey);

 // 6. Plot all signals in one figure
```

```
scf(0);
clf();

// Input x(n)
subplot(2,2,1);
plot2d3(n_axis', x');
xtitle("x(n) - input", "n", "x(n)");
a = gca();
a.data_bounds = [-0.5, -4; 4.5, 3];
xgrid();

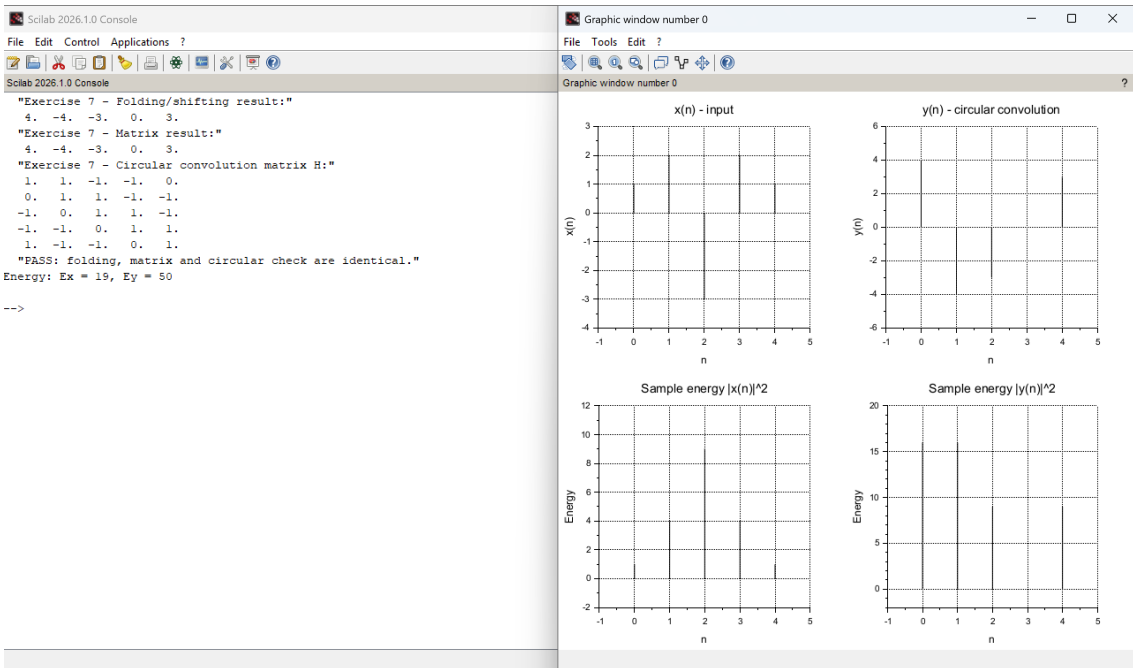
// Circular convolution y(n)
subplot(2,2,2);
plot2d3(n_axis', y_fold');
xtitle("y(n) - circular convolution", "n", "y(n)");
a = gca();
a.data_bounds = [-0.5, -5; 4.5, 5];
xgrid();

// Energy |x(n)|^2
subplot(2,2,3);
plot2d3(n_axis', ex');
xtitle("Sample energy |x(n)|^2", "n", "Energy");
a = gca();
a.data_bounds = [-0.5, -1; 4.5, 11];
xgrid();

// Energy |y(n)|^2
subplot(2,2,4);
plot2d3(n_axis', ey');
xtitle("Sample energy |y(n)|^2", "n", "Energy");
a = gca();
a.data_bounds = [-0.5, -1; 4.5, 19];
xgrid();
```

Listing 2: Mã Scilab cho Exercise 7 – tích chập vòng

6. Kết quả kiểm thử trên Scilab



Hình 2: Exercise 7: kết quả Console

Nhận xét: Phép chập vòng tạo đầu ra có đúng $N = 5$ mẫu trong một chu kỳ; cả hai cách tính đều thu được $[4, -4, -3, 0, 3]$. Kết quả còn được kiểm chứng bằng cách cộng các mẫu chập tuyến tính có chỉ số đồng dư modulo 5.

5 Bàn luận

Qua bài thực hành Lab 4, nhóm đã củng cố được các kiến thức cơ bản về biến đổi Z, miền hội tụ (ROC) và các phương pháp xử lý tín hiệu rời rạc trong môi trường Scilab. Thông qua ba bài tập đầu tiên, nhóm hiểu rõ hơn mối quan hệ giữa đặc điểm của tín hiệu, biểu thức biến đổi Z và miền hội tụ tương ứng.

Bên cạnh đó, việc tìm hiểu các công cụ xử lý âm thanh và ảnh số trong Scilab giúp nhóm tiếp cận những ứng dụng thực tế của xử lý tín hiệu số. Các thao tác như tạo tín hiệu âm thanh, phân tích histogram, cân bằng độ tương phản và làm mờ ảnh cho thấy khả năng ứng dụng của các phép toán tín hiệu trong nhiều lĩnh vực khác nhau.

Đặc biệt, ở Exercise 6 và Exercise 7, nhóm đã triển khai tích chập tuyến tính và tích chập vòng bằng hai phương pháp: gấp–dịch (Folding and Shifting) và phương pháp ma trận (Matrix Method). Kết quả kiểm thử trên Scilab cho thấy các phương pháp đều cho kết quả trùng khớp. Năng lượng tín hiệu đầu ra lần lượt là $E_y = 42$ đối với tích chập tuyến tính và $E_y = 50$ trong một chu kỳ của tích chập vòng.

Thông qua việc xây dựng thuật toán, tính toán và trực quan hóa kết quả, nhóm đã nâng cao khả năng vận dụng kiến thức lý thuyết vào lập trình. Đây là nền tảng quan trọng để tiếp tục nghiên cứu các kỹ thuật xử lý tín hiệu số nâng cao trong những bài thực hành tiếp theo.

6 Tài liệu tham khảo

1. Scilab Enterprises S.A.S. *Scilab for very beginners*.
https://www.scilab.org/sites/default/files/Scilab_beginners.pdf

2. Scilab Online Help. *Signal Processing – Convolution and Modulo Functions*.
https://help.scilab.org/docs/2026.1.0/en_US/conv.html
https://help.scilab.org/docs/2025.0.0/en_US/modulo.html

3. Mã nguồn thực hành Scilab trên GitHub.
https://github.com/neit087/DSP_Scilab.git